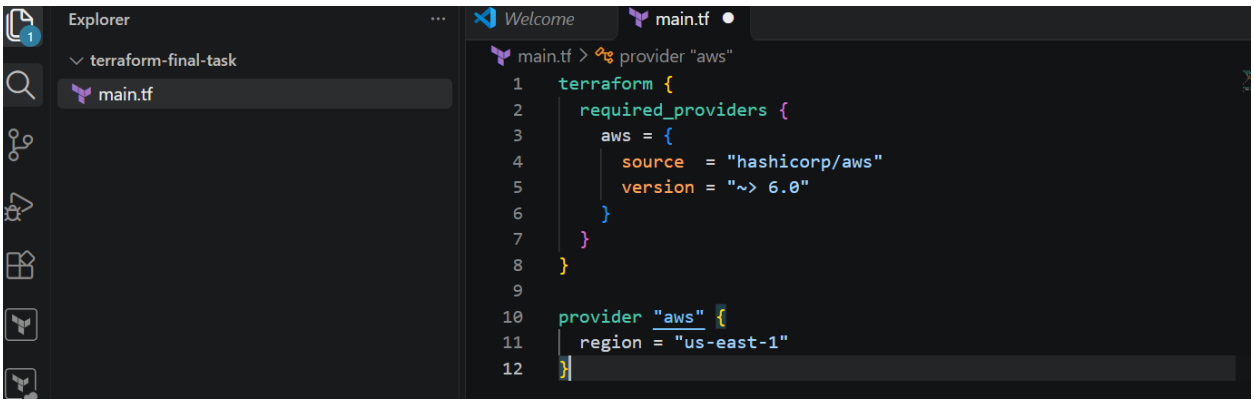
Terraform Final Task:

- **Select New File.**
- **Name it `main.tf`.**
- **Open it and paste this code:**



Add VPC resource

Open main.tf and add this code below the provider block:



this does:

- **Creates a VPC.**
- **Assigns CIDR range `10.30.0.0/16`.**
- **Enables DNS support and hostnames.**

Create Internet Gateway

Add this code below your VPC resource in main.tf:

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Terraform-Final-IGW"
  }
}
```

this does:

- **Creates an Internet Gateway.**
- **Attaches it to the VPC we created in Step 3.**
- **Allows the VPC to connect to the internet when routing is configured.**

<u>Create a public subnet</u>

Add this code below the Internet Gateway resource in main.tf:

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.30.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Terraform-Final-Public-Subnet"
  }
}
```

this does:

- **Creates a subnet inside your VPC.**
- **Assigns CIDR `10.30.1.0/24`.**
- **Uses Availability Zone `us-east-1a`.**
- **Enables automatic public IP assignment.**

<u>Create Route Table</u>

Add this code below your subnet resource in main.tf:

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Terraform-Final-Public-RT"
  }
}
```

this does:

- **Creates a custom route table.**
- **Routes all outbound IPv4 traffic (`0.0.0.0/0`) through the Internet Gateway.**

Now we need to associate the subnet with the custom route table.

Route Table Association

Add this code below your route table resource:

```
resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

this does:

It connects your public subnet to the custom route table, allowing the subnet to use the Internet Gateway route.

Create Security Group

Add this code below the route table association in main.tf:

```
resource "aws_security_group" "web_sg" {
 name        = "Terraform-Final-SG"
 description = "Allow SSH, HTTP and HTTPS"
 vpc_id      = aws_vpc.main.id

 ingress {
  description = "SSH"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
  description = "HTTP"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
```

```
 }


 ingress {
  description = "HTTPS"
  from_port   = 443
  to_port     = 443
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }


 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }


 tags = {
  Name = "Terraform-Final-SG"
 }
}
```

Security note: SSH is currently open to all IPv4 addresses. For better security, we should restrict port 22 to your own public IP before applying.

Security Group is configured with ports 22, 80 and 443.

<u>Create Network Interface</u>

Add this code below your Security Group resource:

```
95  resource "aws_network_interface" "web_eni" {
96    subnet_id       = aws_subnet.public.id
97    private_ips     = ["10.30.1.10"]
98    security_groups = [aws_security_group.web_sg.id]
99
100   tags = {
101     Name = "Terraform-Final-ENI"
102   }
103 }
```

this does:

- **Creates a network interface in your public subnet.**
- **Assigns private IP `10.30.1.10`.**
- **Attaches your Security Group to the interface.**

Network Interface is configured with private IP `10.30.1.10`.

<u>Assign Elastic IP</u>

Add this code below your Network Interface resource:

```
104 resource "aws_eip" "web_eip" {
105   domain                    = "vpc"
106   network_interface         = aws_network_interface.web_eni.id
107   associate_with_private_ip = "10.30.1.10"
108
109   tags = {
110     Name = "Terraform-Final-EIP"
111   }
112 }
```

Tip: Reference files or folders with the agent more context about the

main.tf

this does:

- **Allocates an Elastic IP address.**
- **Associates it directly with your Network Interface.**
- **Uses private IP `10.30.1.10`.**

<u>Create Ubuntu EC2 and install Apache2</u>

Add this code at the bottom of your main.tf:

```
resource "aws_instance" "ubuntu" {
  ami           = "ami-0c7217cdde317cfec"
  instance_type = "t3.micro"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index         = 0
  }

  user_data = <<-EOF
              #!/bin/bash
              apt update -y
              apt install apache2 -y
              systemctl start apache2
              systemctl enable apache2
              echo "<h1>Apache Server Deployed Using Terraform</h1>" > /var/www/html/index.html
              EOF

  tags = {
    Name = "Terraform-Final-Ubuntu"
  }
}
```

this does:

- **Launches an Ubuntu EC2 instance.**
- **Attaches the Network Interface created earlier.**
- **Installs Apache2 automatically using user data.**
- **Starts and enables Apache2.**
- **Creates a sample website.**

The AMI above is an Ubuntu 22.04 AMI commonly used in `us-east-1`; we'll verify it before applying.

All 9 Terraform resource configurations are now added.

Before creating AWS resources, let's validate everything.

Run: terraform init

```
PS C:\Users\india\OneDrive\Desktop\terraform-final-task> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.71.0...
- Installed hashicorp/aws v6.71.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\india\OneDrive\Desktop\terraform-final-task>
```

```
PS C:\Users\india\OneDrive\Desktop\terraform-final-task> terraform fmt
PS C:\Users\india\OneDrive\Desktop\terraform-final-task> terraform validate
Success! The configuration is valid.

PS C:\Users\india\OneDrive\Desktop\terraform-final-task>
```

```
aws_instance.ubuntu: Modifying... [id=i-09ab6b73f5c9d9246]
aws_instance.ubuntu: Still modifying... [id=i-09ab6b73f5c9d9246, 00m10s elapsed]
aws_instance.ubuntu: Still modifying... [id=i-09ab6b73f5c9d9246, 00m20s elapsed]
aws_instance.ubuntu: Still modifying... [id=i-09ab6b73f5c9d9246, 00m30s elapsed]
aws_instance.ubuntu: Still modifying... [id=i-09ab6b73f5c9d9246, 00m40s elapsed]
aws_instance.ubuntu: Still modifying... [id=i-09ab6b73f5c9d9246, 00m50s elapsed]
aws_instance.ubuntu: Modifications complete after 51s [id=i-09ab6b73f5c9d9246]
aws_eip.web_eip: Modifying... [id=eipalloc-0041e5745050d01f6]
aws_eip.web_eip: Modifications complete after 3s [id=eipalloc-0041e5745050d01f6]

Apply complete! Resources: 0 added, 2 changed, 0 destroyed.

Outputs:

elastic_ip = "3.229.217.25"
instance_id = "i-09ab6b73f5c9d9246"
PS C:\Users\india\OneDrive\Desktop\terraform-final-task>
```

Ubuntu EC2

- Instance ID: `i-09ab6b73f5c9d9246`
- Elastic IP: `3.229.217.25`
- Private IP: `10.30.1.10`
- Instance type: `t3.micro`
- Apache2: Installed through user data



Apache Server Deployed Using Terraform

Final Terraform assignment: Completed

All 9 requirements are working:

- VPC and Internet Gateway
- Custom Route Table and subnet association
- Security Group (22, 80, 443)
- Network Interface with private IP
- Elastic IP association
- Ubuntu EC2 instance
- Apache2 installation and website deployment